Video 19 — Structs in Golang (Deep Explanation)

What is a Struct?

A **struct** is a **custom data structure** that groups multiple related fields together. It is Go's way of doing what **classes** do in other languages like Java, Python, or JavaScript.

> Go has NO classes — structs are used instead

File Setup

```
golang-course/
└── 16-structs/
    └── struct.go
```

```go
package main

import "fmt"

func main() {
    // code goes here
}
```

Defining a Struct

```go
type Order struct {
    id        string
    amount    float32
    status    string
    createdAt time.Time
}
```

Breaking it down:

Part	Meaning
`type`	keyword to define a new type
`Order`	name of the struct
`struct`	keyword saying this is a struct
`{ }`	contains all fields
`id string`	field name + field type
`time.Time`	using the built-in time package

- `time.Time` gives **nanosecond precision** — very accurate
- `time` package is automatically imported by VS Code extension

Creating an Instance of a Struct

Way 1 — Full declaration:

```
var myOrder Order = Order{
    id:     "1",
    amount: 50.00,
    status: "received",
}
```

Way 2 — Shorthand (most common):

```
myOrder := Order{
    id:     "1",
    amount: 50.00,
    status: "received",
}
```

Adding field value later using dot notation:

```
myOrder.createdAt = time.Now()  // set after creation
```

Printing the struct:

```
fmt.Println(myOrder)
// Output: {1 50 received 2024-01-01 00:00:00 +0000}
```

Accessing Fields — Dot Notation

```
fmt.Println(myOrder.status)   // received
fmt.Println(myOrder.amount)   // 50
fmt.Println(myOrder.id)       // 1
```

Use `.fieldName` to access any field of a struct.

Struct is a Blueprint — Multiple Instances

Just like a class, a struct is a **blueprint** — you can create multiple independent instances:

```
myOrder1 := Order{
    id:     "1",
    amount: 50.00,
    status: "received",
}

myOrder2 := Order{
    id:     "2",
    amount: 100.00,
    status: "delivered",
}

// Change only myOrder1's status
myOrder1.status = "paid"

fmt.Println(myOrder1)  // status: paid
fmt.Println(myOrder2)  // status: delivered — NOT affected!
```

- Each instance is **completely independent** in memory

- Changing one does NOT affect the other
- This is the power of structs

Zero Values in Structs

If you don't set a field, it gets its **zero value** automatically:

```
myOrder := Order{
    id: "1",
    // amount not set
    // status not set
}

fmt.Println(myOrder.amount)  // 0 — float zero value
fmt.Println(myOrder.status)  // "" — string zero value
```

Field Type	Zero Value
`int`	`0`
`float32`	`0`
`string`	`""`
`bool`	`false`
`time.Time`	zero time

Methods on Structs

Just like classes have methods, structs can have methods attached to them. This is how Go does **Object Oriented Programming**.

Adding a method — Receiver Type:

```
func (o Order) changeStatus(status string) {
    o.status = status
}
```

Breaking down the syntax:

```
func (o Order) changeStatus(status string) {
//    ↑ receiver
//      ↑ struct type
```

Part	Meaning
`(o Order)`	receiver — connects method to Order struct
`o`	name for the struct instance inside method
`Order`	which struct this method belongs to
`changeStatus`	method name
`status string`	parameter

Calling the method:

```
myOrder.changeStatus("confirmed")
```

The Pointer Problem with Methods

```
func (o Order) changeStatus(status string) {
    o.status = status  // changes COPY — original not affected!
}

myOrder.changeStatus("confirmed")
fmt.Println(myOrder.status)  // still "received" — NOT changed!
```

Why? Same reason as pointers lesson — when you pass a struct to a method, Go creates a **copy**. You're modifying the copy, not the original.

Fix — Use Pointer Receiver:

```
func (o *Order) changeStatus(status string) {
//      ↑ pointer receiver
    o.status = status  // changes ORIGINAL — no * needed!
}
```

```
myOrder.changeStatus("confirmed")
fmt.Println(myOrder.status)  // "confirmed" — changed!
```

- Add `*` before the struct name in receiver
- Now `o` points to the **actual struct** not a copy
- No need to dereference with `*` when accessing fields — **Go does it automatically for structs**

When to Use Pointer Receiver vs Normal Receiver

Situation	Use
Modifying struct fields	`*Order` — pointer receiver
Only reading struct fields	`Order` — normal receiver (pointer optional)

```
// Modifying — needs pointer
func (o *Order) changeStatus(status string) {
   o.status = status  // modifying
}

// Reading — pointer optional
func (o Order) getAmount() float32 {
   return o.amount  // just reading
}
```

Constructor Function — Go Style

Go has NO built-in constructors like Java or Python. But we create a **constructor function by convention**:

```
func newOrder(id string, amount float32, status string) *order {
   myOrder := Order{
      id:     id,
      amount: amount,
      status: status,
   }
```

```
    return &myOrder  // return POINTER to struct
}
```

Breaking it down:

Part	Meaning
`newOrder`	convention — start with `new` + struct name
Parameters	same fields as struct
`*Order`	returns a pointer to Order
`&myOrder`	get address of created struct

Convention:

- Start constructor name with `new` → `newOrder`, `newUser`, `newProduct`
- Sometimes written with capital N → `NewOrder` (covered in packages lesson)
- Return a **pointer** to the struct — this is Go convention

Calling the constructor:

```
myOrder := newOrder("1", 50.00, "received")
fmt.Println(myOrder)  // prints the order

// Use dot notation normally — no dereferencing needed
fmt.Println(myOrder.getAmount())  // 50
myOrder.changeStatus("paid")
```

> Even though `myOrder` is a pointer, Go **automatically dereferences** for struct field access — you don't need `*myOrder.field`

Why Return a Pointer from Constructor?

```
// Return value — creates COPY when used
func newOrder(...) Order {
    return myOrder  // copy returned
}
```

```go
// Return pointer — no copy, same instance
func newOrder(...) *Order {
    return &myOrder  // address returned — efficient
}
```

- Returning pointer is more **memory efficient** — no copy created
- The same instance is used everywhere
- This is the **Go convention** for constructors

Anonymous Struct — Inline Struct

When you only need a struct **once** and don't want to define it separately:

```go
language := struct {
    name   string
    isGood bool
}{
    "Golang",
    true,
}

fmt.Println(language)  // {Golang true}
```

Breaking it down:

```go
language := struct {    // define fields inline
    name   string
    isGood bool
}{                      // immediately provide values
    "Golang",           // same ORDER as fields above
    true,
}
```

- No `type` keyword
- No name for the struct
- Fields and values defined together
- Values must be in **same order** as fields
- Similar to object literals in JavaScript

When to use:

- One-time use struct
- Configuration objects
- Quick data grouping

Full Code Example

```
package main

import (
    "fmt"
    "time"
)

// Struct definition
type Order struct {
    id        string
    amount    float32
    status    string
    createdAt time.Time
}

// Constructor function
func newOrder(id string, amount float32, status string) *Order {
    myOrder := Order{
        id:        id,
        amount:    amount,
        status:    status,
        createdAt: time.Now(),
    }
    return &myOrder
}

// Method — modifying (pointer receiver)
func (o *Order) changeStatus(status string) {
    o.status = status
}

// Method — reading (normal receiver)
func (o Order) getAmount() float32 {
    return o.amount
```

```go
}

func main() {

    // Create instance using constructor
    myOrder := newOrder("1", 50.00, "received")
    fmt.Println(myOrder)

    // Access fields
    fmt.Println(myOrder.status)  // received

    // Call method — change status
    myOrder.changeStatus("paid")
    fmt.Println(myOrder.status)  // paid

    // Call method — get amount
    fmt.Println(myOrder.getAmount())  // 50

    // Multiple independent instances
    myOrder2 := newOrder("2", 100.00, "delivered")
    fmt.Println(myOrder2.status)  // delivered — independent

    // Anonymous struct
    language := struct {
        name   string
        isGood bool
    }{
        "Golang",
        true,
    }
    fmt.Println(language)  // {Golang true}
}
```

Running the Program

go run 16-structs/struct.go

Structs vs Classes Comparison

Feature	Classes (Java/Python)	Structs (Go)
Define structure	`class Name { }`	`type Name struct { }`
Create instance	`new ClassName()`	`Name{ }` or constructor func
Methods	Inside class	Outside, with receiver
Constructor	Built-in	Manual function (convention)
Inheritance	✅ Yes	❌ No — uses composition
Pointer receiver	Not needed	Required to modify fields

Summary Table

Concept	Code
Define struct	`type Name struct { field type }`
Create instance	`myOrder := Order{id: "1"}`
Access field	`myOrder.status`
Set field	`myOrder.status = "paid"`
Normal method	`func (o Order) method()`
Pointer method	`func (o *Order) method()`
Constructor	`func newOrder(...) *Order`
Anonymous struct	`struct{ name string }{"Go"}`
Zero value	unset fields get zero values

Key Takeaways

1. Structs are Go's version of **classes** — groups related data together
2. Use `type Name struct { }` to define a struct

3. Create instances using `Name{ field: value }` syntax
4. Access fields using **dot notation** `myOrder.status`
5. Each instance is **completely independent** in memory
6. Unset fields get **zero values** automatically
7. Methods are attached using **receiver types** `(o Order)`
8. Use **pointer receiver** `(o *Order)` when you need to **modify** the struct
9. Go **automatically dereferences** struct pointers — no `*` needed when accessing fields
10. Constructors are just **regular functions** by convention — start with `new`
11. Return **pointer** from constructor — more efficient, no copy
12. **Anonymous structs** for one-time use — no name needed
13. Structs are used **everywhere** in real Go projects — master them well

Let me go through the transcript carefully and check what I may have missed or not explained deeply enough:

Things I Missed or Under-explained

---

1. ❌ Missed — Two Ways of Declaring Instance (Full var syntax)

I showed shorthand but didn't clearly show BOTH ways:

```
// Way 1 — Full declaration
var myOrder Order = Order{
    id: "1",
}

// Way 2 — Shorthand (most common)
myOrder := Order{
    id: "1",
}
```

The instructor specifically showed both and explained you can use either.

---

2. ❌ Missed — Fields are Optional When Creating Instance

The instructor specifically said:

> "There is no requirement that all fields must be filled — if you don't want to add a field you can skip it"

```
myOrder := Order{
    id:     "1",
    status: "received",
    // amount not given — totally fine!
    // createdAt not given — totally fine!
}
```

No error thrown for missing fields — they just get zero values.

---

3. ❌ Missed — The Warning Explanation

When using normal receiver and modifying:

```
func (o Order) changeStatus(status string) {
    o.status = status
}
```

VS Code shows a warning — **"unaddressed write to field status"** — the instructor explained this warning disappears as soon as you add `*` to make it a pointer receiver. I didn't explain this warning at all.

---

4. ❌ Missed — Why No `*` Needed When Accessing Fields on Pointer Receiver

The instructor specifically addressed this question:

> "You might think — we received a pointer, so shouldn't we use `*` to dereference like we did in the pointers lesson?"

```
func (o *Order) changeStatus(status string) {
    // You might expect: (*o).status = status
    // But Go does this automatically for structs!
    o.status = status  // ✅ works directly — no * needed
}
```

Go **automatically dereferences** struct pointers. This is a special behavior specific to structs — I mentioned it briefly but didn't explain the WHY clearly.

---

5. ❌ Missed — Constructor Name Convention Details

The instructor mentioned:

- Convention is to start with `new` → `newOrder`
- Sometimes you'll see **capital N** → `NewOrder`
- Capital vs lowercase has a reason related to **packages** — will be covered later
- For now keeping it lowercase

```
func newOrder(...)  // lowercase — current lesson
func NewOrder(...)  // capital — seen in real code, explained in packages lesson
```

I mentioned this but didn't explain it clearly enough.

---

6. ❌ Missed — Anonymous Struct Value Order is Strict

The instructor specifically said:

> "Values must be in the EXACT same order as fields are defined"

```
language := struct {
   name   string  // field 1
   isGood bool    // field 2
}{
   "Golang",  // must be first — matches name
   true,      // must be second — matches isGood
}

// ❌ Swapping order would cause error or wrong values
```

I mentioned same order but didn't emphasize it was STRICT and what happens if wrong.

---

7. ❌ Missed — The `createdAt` Assignment Style Shown

The instructor showed TWO ways to set `createdAt`:

```
// Way 1 — inside struct literal (doesn't work directly without time.Now())
myOrder := Order{
    createdAt: time.Now(),
}

// Way 2 — after creation using dot notation
myOrder := Order{
    id: "1",
}
myOrder.createdAt = time.Now()  // set afterwards
```

Both approaches were shown and I didn't cover the second way clearly.

---

8. ❌ Missed — Ctrl+Space Autocomplete Tip

The instructor showed that pressing **Ctrl+Space** inside struct literal gives you autocomplete for field names — a practical VS Code tip shown in the transcript.

What I DID Cover Correctly ✅

- Struct definition syntax
- Creating instances
- Dot notation for access
- Multiple independent instances
- Methods with receiver types
- Pointer receiver for modification
- Constructor function pattern
- Returning pointer from constructor
- Anonymous structs
- Zero values
- Comparison with classes

Summary

I covered **most things** but missed these specific details:

1. Both var declaration styles clearly shown
2. Fields being optional at creation
3. The VS Code warning explanation
4. Why `*` not needed for struct field access (auto-dereference)
5. Capital vs lowercase constructor naming and why
6. Strict ordering in anonymous structs
7. Two ways to set `createdAt`
8. Ctrl+Space autocomplete tip